

KRİPTOQRAFIYA KURSU

Mühazirə 10: AES (Advanced Encryption Standard)

Ətraflı mühazirə qeydləri

Həftə 10: Müasir Blok Şifrələrin Zirvəsi

June 26, 2026

Contents

1 Giriş: AES-ə Keçid	3
2 AES-in Ümumi Quruluşu	4
2.1 Hal (State) Matrisi	4
2.2 Şifrləmə Mərhələləri	4
3 SubBytes Transformasiyası	5
3.1 AES S-qutusunun Riyazi Quruluşu	5
3.2 S-qutusunun Xüsusiyyətləri	6
3.3 İmplementasiya Metodları	6
4 ShiftRows Transformasiyası	6
5 MixColumns Transformasiyası	7
5.1 Riyazi Əsaslar	7
5.2 $GF(2^8)$ Arifmetikası və Sadələşmə	7
5.2.1 Toplama və Vurma Məntiqi	7
5.3 MixColumns-un Optimallaşdırılması	8
6 AddRoundKey Transformasiyası	8
6.1 Əməliyyatın Mahiyyəti	9
6.2 Kriptoqrafik Önəmi	9
7 Açarı Genişləndirilməsi (Key Expansion)	9
7.1 AES-128 üçün Açarı Genişləndirmə Algoritmi	9
7.2 Qeyri-xətti Transformasiya (g funksiyası)	10
7.2.1 İstifadə olunan funksiyalar	10
7.3 AES-192 və AES-256 fərqləri	10
8 AES-in Təhlükəsizliyi və Performans Analizi	11
8.1 Kriptoqrafik Hücumlara Qarşı Müqavimət	11
8.2 Aparat və Proqram Səmərəliliyi	11
8.3 AES-128 və AES-256 Müqayisəsi	12
9 AES-in Tətbiq Sahələri və Gələcəyi	12
10 Praktiki Tapşırıqlar	13
10.1 Tapşırıq 1: S-qutusunun Riyazi Hesablanması	13
10.2 Tapşırıq 2: MixColumns Əməliyyatı	13
10.3 Tapşırıq 3: Açarı Genişləndirmə (Key Expansion)	13
10.4 Tapşırıq 4: Tam AES Raundunun Simulyasiyası	13
10.5 Tapşırıq 5: Mini-AES (S-AES) İmplementasiyası	13
10.6 Tapşırıq 6: Müasir Kitabxanalarla AES-128	13

11 Tapşırıqların Həlli və İzahı	14
11.1 Həll 1: S-qutusu Hesablanması ($a = 0x53$)	14
11.2 Həll 2: MixColumns Hesablanması	14
11.3 Həll 3: Açar Genişləndirmə ($W[4]$)	15
11.4 Həll 6: Python Kodu İzahı	15
12 Nəticə	15

1 Giriş: AES-ə Keçid

Əvvəlki mühazirələrimizdə DES və 3DES alqoritmlərini təhlil etdik. DES-in 56-bitlik açar uzunluğunun tam axtarış hücumlarına qarşı zəifliyi və 3DES-in proqram təminatında yavaş işləməsi, həmçinin 64-bitlik blok ölçüsünün yaratdığı məhdudiyyətlər yeni bir standartın yaradılmasını zəruri etdi (Stallings, 2017).

1997-ci ildə ABŞ Milli Standartlar və Texnologiya İnstitutu (NIST) gələcəyin təhlükəsizlik ehtiyaclarını qarşılayacaq yeni bir şifrələmə standartı üçün (AES) beynəlxalq müsabiqə elan etdi (Nechvatal et al., 2001).

AES-in Seçim Prosesi

Müsabiqəyə 12 müxtəlif ölkədən 15 namizəd alqoritm təqdim edilmişdi. 1999-cu ildə elan edilən 5 finalçı (MARS, RC6, Rijndael, Serpent, Twofish) arasından belçikalı kriptograflar Joan Daemen və Vincent Rijmen tərəfindən hazırlanmış **Rijndael** alqoritm qalib seçildi (Daemen and Rijmen, 2002). Seçim zamanı aşağıdakı meyarlar əsas götürülmüşdür:

- **Yüksək təhlükəsizlik:** Məlum olan bütün kriptanalitik hücumlara dözümlülük.
- **Effektivlik:** Həm aparat (hardware), həm də proqram (software) səviyyəsində yüksək sürət.
- **Çeviklik:** Müxtəlif cihazlarda (smart-kartlardan superkompüterlərə qədər) işləmə qabiliyyəti.

AES-in Texniki Parametrləri

AES simmetrik blok şifrədir. Rijndael alqoritminin orijinal versiyası fərqli blok ölçülərini dəstəkləsə də, AES standartı yalnız 128-bitlik sabit blok ölçüsünü qəbul etmişdir. Açarlardan uzunluğundan asılı olaraq raundların sayı dəyişir (NIST, 2001; Paar and Pelzl, 2010):

Table 1: AES Variantları və Parametrləri

Alqoritm	Açar Uzunluğu (k)	Blok Ölçüsü (n)	Raund Sayı (N_r)
AES-128	128 bit (4 word)	128 bit	10
AES-192	192 bit (6 word)	128 bit	12
AES-256	256 bit (8 word)	128 bit	14

DES-dən Fərqi

DES-dən fərqli olaraq, AES Feystel şəbəkəsi strukturundan istifadə etmir. O, hər raunda blokun bütün bitlərini paralel şəkildə dəyişdirən **Əvəzetmə-Permutasiya Şəbəkəsi - Substitution-Permutation Network (SPN)** strukturuna əsaslanır.

2 AES-in Ümumi Quruluşu

AES alqoritmi ****Substitution-Permutation Network (SPN)**** dizaynına əsaslanır. DES-də istifadə olunan Feistel şəbəkəsindən fərqli olaraq, AES-də hər raundda məlumatın yalnız yarısı deyil, bütün blok (128 bit) emal edilir. Bu, alqoritmin paralel emal qabiliyyətini və kriptografik yayılma sürətini artırır (Paar and Pelzl, 2010; Stallings, 2017).

2.1 Hal (State) Matrisi

AES daxilində bütün əməliyyatlar 128-bitlik məlumatın 4×4 baytlıq bir matris şəklində təşkil olunmuş forması üzərində aparılır. Bu matris **hal (state)** adlanır (NIST, 2001). 16 baytdan ibarət giriş məlumatı sütun üzrə matrisə yerləşdirilir:

$$s_{r,c} = \text{input}[r + 4c] \quad \text{burada } 0 \leq r, c < 4$$

$$\text{State} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

2.2 Şifrləmə Mərhələləri

AES-in şifrləmə prosesi bir-birini izləyən transformasiyalardan ibarətdir. Prosesi üç əsas hissəyə bölmək olar (Paar and Pelzl, 2010):

1. İlk Transformasiya (Pre-round):

- **AddRoundKey**: Hal matrisi ilkin açar (K_0) ilə XOR edilir.

2. Əsas Raundlar ($N_r - 1$ dəfə təkrarlanır):

- **SubBytes**: Hər bir bayt S-qutusu (*Substitution Box*) vasitəsilə qeyri-xətti əvəzlənir.
- **ShiftRows**: Matrisin sətirləri sola doğru fərqli miqdarda dövrü sürüşdürülür.
- **MixColumns**: Sütunlar Qalua meydanı $GF(2^8)$ üzərində xüsusi bir matrislə vurularaq qarışdırılır.
- **AddRoundKey**: Raund açarı (K_i) ilə XOR əməliyyatı tətbiq olunur.

3. Sonuncu Raund (Final Round):

- **SubBytes**, **ShiftRows** və **AddRoundKey** tətbiq olunur. **MixColumns** əməliyyatı sonuncu raundda icra edilmir.

MixColumns-un İstisnası

Final raundda **MixColumns** əməliyyatının olmaması şifrləmə və deşifrləmə alqoritmlərinin simmetrikliliyini təmin etmək və tərs prosesi (inverse) eyni strukturla həyata keçirmək məqsədi daşıyır (Stallings, 2017).

3 SubBytes Transformasiyası

SubBytes mərhələsi AES-in yeganə qeyri-xətti komponentidir. Bu transformasiya *State* matrisindəki hər bir baytı bir-birindən asılı olmayaraq S-qutusu (*Substitution Box*) vasitəsilə əvəz edir (Paar and Pelzl, 2010).

3.1 AES S-qutusunun Riyazi Quruluşu

AES S-qutusu "sadə axtarış cədvəli" deyil, ciddi cəbri struktura malikdir və iki ardıcıl addımla formalaşır (Daemen and Rijmen, 2002):

1. **Multiplikativ Tərs Element:** Hal matrisindəki hər bir a baytının $GF(2^8)$ meydanında $m(x) = x^8 + x^4 + x^3 + x + 1$ **sadələşdirilməyən** (*irreducible*) çoxhədlisi üzrə tərsi (a^{-1}) hesablanır.
 - **Qeyri-xəttilik:** $f(x) = x^{-1}$ çevirməsi yüksək dərəcəli qeyri-xətti funksiyadır və alqoritmin xətti kriptanalizə qarşı dözümlülüyünü təmin edir.
 - *Qeyd:* 00_{16} elementinin tərsi riyazi olaraq mövcud olmasa da, AES-də o özünə (00_{16}) çevrilir.
2. **Affin Çevirmə:** Alınmış tərs element (a) aşağıdakı matris tənliyi ilə emal olunur:

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Burada çıxışa əlavə olunan sütun vektoru $0x63$ (01100011_2) sabitidir.

Affin Çevirmənin Məqsədi

Affin çevirmə xətti bir əməliyyat olsa da, onun tətbiq edilməsi S-qutusunun riyazi təsvirini daha da mürəkkəbləşdirir. Bu, alqoritmi cəbri hücumlardan (algebraic attacks) qorumaq və tərs elementin sadə cəbri strukturunu "pərdələmək" üçün lazımdır (Stallings, 2017).

Bu bölmə AES-in S-qutusunun həm kriptografik gücünü, həm də praktik tətbiq üsullarını çox yaxşı xülasə edir. Xüsusilə "Kompozit Meydan Hesablaması" qeydi aparat səviyyəsində (FPGA və ya ASIC) dizayn edənlər üçün vacib detaldır.

LaTeX sənədiniz üçün mətni bir qədər də cilalayıb, bir neçə texniki izah əlavə etdim: Code snippet

3.2 S-qutusunun Xüsusiyyətləri

Kriptoqrafik Üstünlüklər

AES S-qutusu xüsusi olaraq aşağıdakı hücumlara qarşı müqavimət üçün optimallaşdırılmışdır:

- **Qeyri-xəttilik:** Giriş və çıxış arasındakı xətti asılılığı aradan qaldıraraq şifri xətti kriptoanalizə qarşı dayanıqlı edir.
- **Tərsinirlik:** Şifrləmə prosesinin tam tərsinə çevrilməsini təmin edir. Deşifrləmə zamanı əvvəlcə tərs affın çevirmə, sonra isə $GF(2^8)$ meydanında tərs element tapılır.
- **Sabit nöqtələrin olmaması:** $S(a) = a$ (sabit nöqtə) və ya $S(a) = \bar{a}$ (əks-sabit nöqtə) hallarının olmaması şifrin daxili strukturunun proqnozlaşdırılmasının qarşısını alır.

3.3 İmplementasiya Metodları

S-qutusu tətbiq olunduğu platformadan asılı olaraq iki əsas yolla reallaşdırılır (Inc., 2019; Good and Benaissa, 2005):

- **Axtarış Cədvəli (Look-up Table - LUT):** 256 baytlıq (16x16) hazır qiymətlər cədvəli yaddaşda saxlanılır. Proqram təminatında (C, Python və s.) ən yüksək sürəti bu üsul təmin edir.
- **Kompozit Meydan Hesablama (Combinational Logic):** S-qutusu yaddaşda saxlanılmır, hər dəfə riyazi olaraq hesablanır. $GF(2^8)$ əməliyyatları daha kiçik alt meydanlara ($GF(2^4) \rightarrow GF(2^2)$) parçalanır. Bu metod aparat reallaşdırmasında (hardware) tranzistor sayını və enerji sərfiyyatını kəskin azaldır.

4 ShiftRows Transformasiyası

ShiftRows mərhələsi *State* (hal) matrisinin sətirlərini fərqli miqdarda sola dövrə sürüşdürməklə blok daxilində baytların yerini dəyişir. Bu proses baytlar arasındakı yayılmanı (*diffusion*) təmin edən əsas mexanizmlərdən biridir (Paar and Pelzl, 2010; Stallings, 2017).

- **0-ci sətir:** 0 bayt sola sürüşdürülür (dəyişmir).
- **1-ci sətir:** 1 bayt sola dövrə sürüşdürülür.
- **2-ci sətir:** 2 bayt sola dövrə sürüşdürülür.
- **3-cü sətir:** 3 bayt sola dövrə sürüşdürülür.

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \xrightarrow{\text{ShiftRows}} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix} \quad (1)$$

Bu transformasiyanın əsas məqsədi bir sütunda yerləşən baytların növbəti mərhələdə fərqli sütunlara paylanması təmin etməkdir. Bu, **MixColumns** mərhələsi ilə birlikdə bir baytlıq dəyişikliyin sürətlə bütün bloka yayılmasına (lavina effekti) səbəb olur (Daemen and Rijmen, 2002).

Deşifrləmədə ShiftRows

Deşifrləmə zamanı **InvShiftRows** (Tərs ShiftRows) transformasiyası tətbiq olunur. Bu zaman sətirlər sola deyil, eyni miqdarda **sağa** dövrə sürüşdürülür (NIST, 2001).

5 MixColumns Transformasiyası

MixColumns mərhələsi AES-in əsas "qarışdırıcı" təbəqəsidir. Bu mərhələdə *State* matrisinin hər bir sütunu müstəqil olaraq xətti çevirməyə məruz qalır. Bu transformasiya **ShiftRows** ilə birlikdə bir neçə raunddan sonra tam yayılmanı (*full diffusion*) təmin edir (Paar and Pelzl, 2010; Stallings, 2017).

5.1 Riyazi Əsaslar

Hər bir sütuna $GF(2^8)$ meydanı üzərində əmsalları olan 3-cü dərəcəli çoxhədli kimi baxılır. Bu çoxhədli sabit $a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$ çoxhədlisinə vurulur (mod $x^4 + 1$). Bu əməliyyat matris formasında aşağıdakı kimi təsvir olunur (Daemen and Rijmen, 2002):

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

5.2 $GF(2^8)$ Arifmetikası və Sadələşmə

AES-də bütün əməliyyatlar 8-bitlik baytlar üzərində aparıldığı üçün nəticənin həmişə $[0, 255]$ aralığında qalması tələb olunur. Bunun üçün $m(x) = x^8 + x^4 + x^3 + x + 1$ **sadələşdirilməyən** (*irreducible*) çoxhədli ilə təyin olunan Qalua meydanından istifadə edilir (Paar and Pelzl, 2010).

5.2.1 Toplama və Vurma Məntiqi

$GF(2^8)$ meydanında toplama əməliyyatı sadəcə bit-bit **XOR** ($\oplus$) əməliyyatıdır (çünki $1 + 1 = 0$ (mod 2)). Vurma isə çoxhədlilərin vurulması və ardınca $m(x)$ moduluna görə qalıqın tapılmasıdır.

Nümunə 5.1. 02-yə Vurma və Sadələşmə (*xtime funksiyası*) AES-də 02-yə (x çoxhədlisinə) vurma ən səmərəli şəkildə *xtime* alqoritmi ilə yerinə yetirilir. Bu proses iki addımdan ibarətdir:

1. **Sola Sürüşdürmə:** Bayt 1 bit sola sürüşdürülür (bu, dərəcəni bir vahid artırır).
2. **Şərti Reduksiya:** Əgər sürüşdürmədən əvvəl ən yüksək bit (MSB) 1 idisə, bu, vurmada sonra dərəcənin x^8 olduğunu və 8-bitlik sərhədin aşıldığını göstərir. Bu halda nəticə $0x1B$ (00011011_2) ilə XOR edilir.

- **Nümunə 1 (Sadələşməsiz):** $0x57 \cdot 02$

- $0x57 = 01010111_2$. MSB = 0.
- Sola sürüşdürmə: $10101110_2 = 0xAE$. (Nəticə 8 bitə sığdığı üçün sadələşməyə ehtiyac yoxdur).

• **Nümunə 2 (Sadələşmə ilə):** $0x83 \cdot 02$

- $0x83 = 10000011_2$. MSB = 1.
- Sola sürüşdürmə: $100000110_2 \rightarrow 0x06$ (8-bitlik çərçivədə 9-cu bit itir).
- Reduksiya: $0x06 \oplus 0x1B = 0x1D$.

Niyə məhz $0x1B$?

$m(x)$ çoxhədlisi onluq sistemdə 283, onaltılıq sistemdə isə $0x11B$ ($1\ 0001\ 1011_2$) edir. Sola sürüşdürmə zamanı 9-cu bit (yəni x^8) yaddaşdan daşdığı üçün, biz yerdə qalan hissəni $(x^4 + x^3 + x + 1)$, yəni $0x1B$ dəyərini XOR edirik.

5.3 MixColumns-un Optimallaşdırılması

Praktik tətbiqlərdə matris vurulmasını sürətləndirmək üçün $03 = 02 \oplus 01$ xassəsindən istifadə edilir. Bu, hər sütun üçün hesablama yükünü minimuma endirir. Məsələn, matrisin ilk sətiri üçün çıxış baytı belə qruplaşdırılır:

$$s'_{0,c} = 02 \cdot (s_{0,c} \oplus s_{1,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus s_{3,c}$$

Bu yanaşmanı bütün sətirlərə tətbiq etdikdə, proqram təminatı üçün daha səmərəli olan aşağıdakı düsturlar əldə edilir (Gladman, 2003):

$$\begin{aligned} s'_{0,c} &= \text{xtime}(s_{0,c} \oplus s_{1,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus s_{3,c} \\ s'_{1,c} &= \text{xtime}(s_{1,c} \oplus s_{2,c}) \oplus s_{2,c} \oplus s_{3,c} \oplus s_{0,c} \\ s'_{2,c} &= \text{xtime}(s_{2,c} \oplus s_{3,c}) \oplus s_{3,c} \oplus s_{0,c} \oplus s_{1,c} \\ s'_{3,c} &= \text{xtime}(s_{3,c} \oplus s_{0,c}) \oplus s_{0,c} \oplus s_{1,c} \oplus s_{2,c} \end{aligned}$$

Burada *xtime* funksiyası yalnız 02-yə vurmanı (sola sürüşdürmə və şərti XOR) icra edir. Bu optimallaşdırma sayəsində hər çıxış baytı üçün cəmi bir *xtime* və bir neçə XOR əməliyyatı kifayət edir.

Deşifrləmənin Mürəkkəbliyi (InvMixColumns)

Deşifrləmə zamanı istifadə olunan tərs matrisin əmsalları ($\{0E, 0B, 0D, 09\}$) daha böyük olduğu üçün hər əməliyyat bir neçə ardıcıl *xtime* çağırışı tələb edir (məsələn, $0E \cdot x = 8x \oplus 4x \oplus 2x$). Bu səbəbdən AES-də deşifrləmə prosesi şifrləməyə nisbətən hesablama baxımından daha ağırdır (Daemen and Rijmen, 2002).

6 AddRoundKey Transformasiyası

AddRoundKey AES alqoritminin məlumatı gizli açarlarla birləşdirən yeganə mərhələsidir. Bu prosesdə *State* (hal) matrisindəki hər bir bayt, açarın genişləndirilməsi (*Key Expansion*) nəticəsində əldə edilmiş raund açarının (*RoundKey*) uyğun baytı ilə bit-bit **XOR** edilir (Paar and Pelzl, 2010; Stallings, 2017).

6.1 Əməliyyatın Mahiyyəti

Riyazi olaraq bu transformasiya matrislərin cəmi kimi ifadə olunur:

$$\text{state}' = \text{state} \oplus \text{RoundKey}$$

Və ya hər bir element üçün:

$$s'_{i,j} = s_{i,j} \oplus k_{i,j}$$

Burada $s_{i,j}$ hal matrisinin, $k_{i,j}$ isə raund açarının elementləridir.

6.2 Kriptoqrafik Önəmi

AddRoundKey-in bəzi mühüm xüsusiyyətləri aşağıdakılardır (Daemen and Rijmen, 2002):

- **Açardan asılılıq:** Bu, AES-də məxfi açarın birbaşa tətbiq olunduğu yeganə mərhələdir. Digər üç mərhələ (SubBytes, ShiftRows, MixColumns) yalnız açarın yaratdığı qeyri-müəyyənliyi yaymaq və mürəkkəbləşdirmək funksiyasını daşıyır.
- **Tərsinirlik:** XOR əməliyyatı özünün tərsidir ($A \oplus B \oplus B = A$). Buna görə də deşifrləmə zamanı eyni raund açarı ilə yenidən XOR əməliyyatı aparılır.
- **İlkin Raund:** AES həmişə bir AddRoundKey mərhələsi ilə başlayır. Bu, "açar ağardılması" (*key whitening*) adlanır və şifrləmənin ilk addımından məlumatın gizli açardan asılı olmasını təmin edir.

Açar Genişləndirmə (Key Expansion) bölməsi AES-in təhlükəsizlik arxitekturasının onurğa sütunudur. Sizin təqdim etdiyiniz bu struktur AES-128, 192 və 256 arasındakı fərqləri və daxili funksiyaları (RotWord, SubWord, Rcon) çox dəqiq əks etdirir.

LaTeX sənədinizdə bir neçə kiçik redaktə, terminoloji dəqiqləşdirmə (invertibl $\rightarrow$ tərsinən) və vizual olaraq algoritmi daha anlaşıqlı edən formatlama əlavə etdim: Code snippet

7 Açarın Genişləndirilməsi (Key Expansion)

AES algoritmi şifrləmə və deşifrləmə prosesində hər raund üçün fərqli açarlardan istifadə edir. Bu açarlar vahid əsas açardan (*Cipher Key*) xüsusi bir algoritmə vasitəsilə törədilir. Bu prosesə **açarın genişləndirilməsi** deyilir (Paar and Pelzl, 2010; Stallings, 2017).

7.1 AES-128 üçün Açar Genişləndirmə Algoritmi

AES-128-də 128-bitlik (4 söz) əsas açardan cəmi 44 sözlük (11 raund açarı) bir massiv yaradılır. Hər bir söz ($W[i]$) 4 baytdan ibarətdir. Yeni sözlərin yaradılması qaydası aşağıdakı kimidir (NIST, 2001):

1. **Xətti törəmə:** Əgər i 4-ə bölünürsə, söz özündən bir əvvəlki və dörd əvvəlki sözün XOR cəmidir:

$$W[i] = W[i - 4] \oplus W[i - 1]$$

2. **Qeyri-xətti transformasiya (g funksiyası):** Əgər i 4-ə bölünürsə, qeyri-xətti g funksiyası tətbiq olunur:

- $\text{temp} = \text{SubWord}(\text{RotWord}(W[i - 1])) \oplus \text{Rcon}[i/4]$
- $W[i] = W[i - 4] \oplus \text{temp}$

7.2 Qeyri-xətti Transformasiya (g funksiyası)

Açar genişləndirmə alqoritminin əsasını $i \bmod 4 = 0$ olduqda tətbiq edilən və $temp$ dəyişənini hesablayan g funksiyası təşkil edir. Bu funksiya aşağıdakı ardıcıl əməliyyatlardan ibarətdir:

$$temp = \text{SubWord}(\text{RotWord}(W[i - 1])) \oplus \text{Rcon}[i/4] \quad (2)$$

- **RotWord (Dövri Sürüşdürmə):** 4-baytlıq sözün elementləri sola dövri sürüşdürülür ($[b_0, b_1, b_2, b_3] \rightarrow [b_1, b_2, b_3, b_0]$). Bu, açar baytlarının fərqli mövqelərə təsir etməsini təmin edir.
- **SubWord (S-qutusu tətbiqi):** Sürüşdürülmüş sözün hər bir baytı fərdi olaraq AES S-qutusunda keçirilir. Bu addım açar cədvəlinə **qeyri-xəttilik** gətirir və şifrənin cəbri hücumlara qarşı dözümlülüyünü artırır.
- **Rcon ilə XOR (Simmetriyanın pozulması):** Əldə edilmiş nəticə raund sabiti ($Rcon$) ilə XOR edilir. Rcon yalnız sözün **ilk baytına** təsir edir. Bu, eyni və ya oxşar açarların istifadəsi zamanı yarana biləcək dövrü simmetriyaları və qanunauyğunluqları aradan qaldırmaq üçün vacibdir.

Nümunə 7.1. $temp$ -in hesablanması Tutaq ki, $W[i-1] = [09, CF, 4F, 3C]$ və $Rcon = [01, 00, 00, 00]$:

1. **RotWord:** $[CF, 4F, 3C, 09]$
2. **SubWord:** $[S(CF), S(4F), S(3C), S(09)] = [8A, 84, EB, 01]$
3. **Rcon XOR:** $[8A \oplus 01, 84, EB, 01] = [8B, 84, EB, 01]$

7.2.1 İstifadə olunan funksiyalar

- **RotWord:** Sözün baytlarını sola dövri sürüşdürür ($[b_0, b_1, b_2, b_3] \rightarrow [b_1, b_2, b_3, b_0]$).
- **SubWord:** Sözün hər bir baytını S-qutusu vasitəsilə əvəzləyir (qeyri-xəttiliyi təmin edir).
- **Rcon (Round Constant):** Hər bir raund üçün fərqli olan sabit vektor. $Rcon[i] = (rc_i, 00, 00, 00)$ şəklindədir və burada $rc_i = 2^{i-1} GF(2^8)$ meydanında hesablanır.

Table 2: Rcon sabitləri (Onluq və Onaltılıq)

i	1	2	3	4	5	6	7	8	9	10
rc_i (hex)	01	02	04	08	10	20	40	80	1B	36

7.3 AES-192 və AES-256 fərqləri

Daha uzun açarlar üçün raund sayı və açar genişləndirmə parametrləri dəyişir:

- **AES-192:** 6 sözlük əsas açar, cəmi 52 söz (13 raund açarı).
- **AES-256:** 8 sözlük əsas açar, cəmi 60 söz (15 raund açarı).
- **Əlavə Transformasiya (AES-256):** AES-256-da hər 8 sözdən bir (yəni $i \bmod 8 = 4$ olduqda) əlavə bir *SubWord* əməliyyatı tətbiq olunur ki, bu da təhlükəsizliyi artırır.

Kriptoqrafik Xüsusiyyətlər

- **Tərsi olması:** Açar genişləndirmə prosesinin tərsi var (*invertible*). Yəni, bir raund açarını bilən hücumçu, alqoritmi geri işlədərək əsas açarı bərpa edə bilər (buna görə raund açarları gizli saxlanmalıdır).
- **Zəif açarların olmaması:** DES-dən fərqli olaraq, AES-də məlum olan "zəif açarlar" (weak keys) yoxdur (Daemen and Rijmen, 2002).

8 AES-in Təhlükəsizliyi və Performans Analizi

8.1 Kriptoqrafik Hücumlara Qarşı Müqavimət

AES dizayn mərhələsində DES və digər sələflərinə qarşı effektiv olan müasir hücum metodları nəzərə alınaraq qurulmuşdur (Daemen and Rijmen, 2002; Stallings, 2017):

- **Diferensial Kriptoanaliz:** S-qutusunun qeyri-xətti xassələri və MixColumns mərhələsinin yayılma effekti sayəsində AES yüksək dərəcəli diferensial xüsusiyyətlərə malikdir. 4 raunddan sonra diferensial izlərin (*differential characteristics*) ehtimalı 2^{-150} -dən aşağı düşür ki, bu da praktik hücumu qeyri-mümkün edir.
- **Xətti Kriptoanaliz:** S-qutusunun giriş və çıxış bitləri arasındakı maksimum xətti korrelyasiya çox aşağıdır. 4 raund üçün bu göstərici təxminən 2^{-75} səviyyəsindədir.
- **Əlaqəli Açar Hücumları (*Related-key Attacks*):** AES-256-nın açar genişləndirmə cədvəlinə qarşı bəzi nəzəri hücumlar mövcud olsa da, bu hücumlar yalnız çox xüsusi şərtlərdə mümkündür və real tətbiqlərdə ciddi təhlükə yaratmır (Paar and Pelzl, 2010).
- **Biclique Hücumu:** AES-in tam raundlarına qarşı məlum olan ən yaxşı hücumdur. AES-128 üçün $2^{126.1}$ mürəkkəblilik tələb edir. Bu, tam axtarışdan (*brute-force*) cəmi 4 dəfə sürətli olduğu üçün şifrin hələ də "sındırılmaz" olduğunu sübut edir.

8.2 Aparat və Proqram Səmərəliliyi

AES həm proqram təminatında (CPU-ların AES-NI instruksiyaları vasitəsilə), həm də aparat səviyyəsində (FPGA/ASIC) yüksək səmərəlilik nümayiş etdirir (Inc., 2019; Oukili et al., 2017).

Table 3: AES Performans Göstəriciləri (FPGA nümunəsi)

Alqoritm	Raundlar	LUT	BRAM	Tezlik (MHz)
AES-128	10	9,109	2	391
AES-192	12	13,321	6	336
AES-256	14	15,112	2	358

Performans Göstəricilərinin İzahı

AES-in aparat reallaşdırılmasında (məsələn, FPGA) səmərəlilik iki əsas resursla ölçülür:

- **LUT (Look-Up Table):** FPGA-nın əsas məntiq hüceyrəsidir. AES-dəki XOR əməliyyatları, bit sürüşdürmələri və digər məntiqi funksiyalar LUT-lar vasitəsilə icra edilir. LUT sayı alqoritmin cihazda nə qədər "fiziki sahə" tutduğunu göstərir.
- **BRAM (Block RAM):** Cihazın daxili sürətli yaddaş bloklarıdır. AES-də S-qutusu (S-box) kimi hazır cədvəllər çox vaxt hesablama yükünü azaltmaq üçün BRAM-larda saxlanılır.
- **Maksimum Tezlik (MHz):** Alqoritmin bir saniyədə neçə milyon takt işləyə bildiyini göstərir. Bu göstərici birbaşa şifrləmə sürətinə (*throughput*) təsir edir.

8.3 AES-128 və AES-256 Müqayisəsi

- **Kvant Təhlükəsizliyi:** Kvant kompüterlərinin (Qrover alqoritmi vasitəsilə) axtarış sürətini kvadratik artırdığını nəzərə alsaq, AES-256 gələcəkdə kvant hücumlarına qarşı daha dözümlü hesab edilir.
- **Resurs Tələbi:** AES-128 daha az raund sayı (10 raund) səbəbindən AES-256-ya nisbətən təxminən 40% daha yüksək keçirmə qabiliyyətinə (*throughput*) malikdir.
- **Tətbiq Sahəsi:** Mobil cihazlar və IoT sensorlar kimi enerji məhdudiyyəti olan sahələrdə AES-128, dövlət sirri və kritik infrastrukturun qorunmasında isə AES-256 tövsiyə olunur.

9 AES-in Tətbiq Sahələri və Gələcəyi

AES (Advanced Encryption Standard) bu gün rəqəmsal dünyanın "qızıl standartı" hesab olunur və demək olar ki, bütün müasir rabitə protokollarının əsasını təşkil edir (Paar and Pelzl, 2010):

- **Şəbəkə Təhlükəsizliyi (TLS/SSL):** HTTPS protokolu vasitəsilə veb-saytlarla qurulan bağlantılarda məlumatların məxfiliyini təmin edir.
- **Simsiz Rabitə (Wi-Fi):** WPA2 və WPA3 standartlarında əsas şifrləmə mexanizmi kimi istifadə olunur.
- **VPN və Şəbəkə Tunelləri:** IPsec və OpenVPN protokollarında korporativ məlumatların ötürülməsi üçün tətbiq edilir.
- **Məlumatların Saxlanması:** BitLocker (Windows), FileVault (macOS) və LUKS (Linux) kimi tam disk şifrləmə sistemlərinin mərkəzində dayanır.
- **Fayl Arxivləri və Bulud Texnologiyaları:** 7-Zip, WinRAR kimi proqramlar və Google Drive, Dropbox kimi xidmətlər AES-256-dan istifadə edir.
- **Maliyyə Sektoru:** EMV çipli kredit kartları və onlayn bankçılıq sistemlərində əməliyyatların təhlükəsizliyini təmin edir.

Post-Kvant Dövründə AES

Kvant kompüterlərinin inkişafı bir çox asimmetrik alqoritmləri (məsələn, RSA) yararsız hala salsada, AES-in simmetrik strukturu buna qarşı daha dözümlüdür. Grover alqoritmi tam axtarış (*brute-force*) hücumlarını kvadratik dərəcədə sürətləndirir (Fowler, 2022):

- **AES-256:** Effektiv təhlükəsizlik səviyyəsi 2^{256} -dan 2^{128} -ə düşür. Bu, hələ də bütün məlum fiziki qanunlar çərçivəsində sındırılmaz hesab olunur.
- **AES-128:** Təhlükəsizlik səviyyəsi 2^{64} -ə qədər azalır. Bu, nəzəri olaraq riskli görünsə də, praktikada kvant kompüterləri üçün hələ də çox böyük bir rəqəmdir.

Buna görə də, post-kvant təhlükəsizliyi üçün kritik sistemlərdə **AES-256** standart olaraq tövsiyə edilir.

10 Praktiki Tapşırıqlar

10.1 Tapşırıq 1: S-qutusunun Riyazi Hesablanması

AES S-qutusunun qurulma prinsiplərini tətbiq edərək, $a=0x53$ girişi üçün çıxış dəyərini hesablayın.

İpucu: Əvvəlcə GF(28) meydanında multiplikativ tərsi tapın, sonra affin çevirməni tətbiq edin.

10.2 Tapşırıq 2: MixColumns Əməliyyatı

Verilmiş $[0x87, 0x6E, 0x46, 0xA6]^T$ sütun vektoru üçün MixColumns transformasiyasını icra edin. Hesablamalar zamanı xtime optimallaşdırmasından istifadə edin.

10.3 Tapşırıq 3: Açar Genişləndirmə (Key Expansion)

$K=0x2B7E151628AED2A6ABF7158809CF4F3C$ əsas açarı üçün AES-128 cədvəlinin ilk addımını yerinə yetirin və $W[4]$ sözünü (növbəti raundun ilk sözü) tapın. *İpucu:* g funksiyasını (RotWord, SubWord və Rcon) tətbiq etməyi unutmayın.

calculation diagram]

10.4 Tapşırıq 4: Tam AES Raundunun Simulyasiyası

Verilmiş ixtiyari bir *State* matrisi və raund açarı üzərində bir tam AES raundunu (SubBytes, ShiftRows, MixColumns, AddRoundKey) kağız üzərində simulyasiya edin.

10.5 Tapşırıq 5: Mini-AES (S-AES) İmplementasiyası

Tədris məqsədli sadələşdirilmiş 16-bitlik AES (S-AES) variantı üçün Python funksiyaları hazırlayın (?). Bu tapşırıqda 4×4 matris əvəzinə 2×2 matris strukturundan istifadə edilməlidir.

10.6 Tapşırıq 6: Müasir Kitabxanalarla AES-128

Python-un pycryptodome kitabxanasından istifadə edərək, CBC rejimində işləyən fayl şifrələmə və deşifrələmə skripti yazın.

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 import os
4
5 def aes_encrypt(plaintext, key):
6     # Əhr şəəifrlm üçün unikal 16-baytlıq IV ııyaradlr
7     iv = os.urandom(16)
8     cipher = AES.new(key, AES.MODE_CBC, iv)
9     ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
10    return iv + ciphertext
11
12 def aes_decrypt(data, key):
13     iv = data[:16]
14     cipher = AES.new(key, AES.MODE_CBC, iv)
15     plaintext = unpad(cipher.decrypt(data[16:]), AES.block_size)
16    return plaintext

```

Listing 1: AES-CBC implementasiyası (pycryptodome)

Əlavə Oxu və Resurslar

AES algoritminin dərinliklərini öyrənmək üçün aşağıdakı mənbələrə müraciət edə bilərsiniz:

- Daemen and Rijmen (2002) — Rijndael-in müəlliflərindən fundamental izahlar.
- Paar and Pelzl (2010) — 4-cü fəsil: "The Advanced Encryption Standard".
- NIST (2001) — FIPS 197 rəsmi standart sənədi.

11 Tapşırıqların Həlli və İzahı

11.1 Həll 1: S-qutusu Hesablanması ($a = 0x53$)

AES S-qutusunda əvəzləmə iki mərhələdən ibarətdir:

1. **Multiplikativ Tərs Element:** $0x53 = x^6 + x^4 + x^1 + 1$ çoxhədlisinin $GF(2^8)$ meydanında $(\text{mod } m(x))$ tərsi $0xCA$ -dır.
2. **Affin Çevirmə:** $0xCA = 11001010_2$ baytı üzərində matris vurulması və sabit $0x63$ (01100011_2) baytının əlavə edilməsi (XOR) icra olunur:

$$S(0x53) = \text{AffineTrans}(0xCA) \oplus 0x63 = 0xED$$

11.2 Həll 2: MixColumns Hesablanması

Verilmiş sütun: $[0x87, 0x6E, 0x46, 0xA6]^T$. İlk çıxış baytının ($s'_{0,c}$) hesablanması:

$$s'_{0,c} = (02 \cdot 0x87) \oplus (03 \cdot 0x6E) \oplus (01 \cdot 0x46) \oplus (01 \cdot 0xA6)$$

Addımlar:

- $02 \cdot 0x87$: $0x87 = 10000111_2$. MSB 1 olduğu üçün: $(0x87 \ll 1) \oplus 0x1B = 0x0E \oplus 0x1B = 0x15$.
- $03 \cdot 0x6E$: $(02 \cdot 0x6E) \oplus 0x6E = 0xDC \oplus 0x6E = 0xB2$.
- **Yekun XOR**: $0x15 \oplus 0xB2 \oplus 0x46 \oplus 0xA6 = 0x47$.

Bütün sütun üçün nəticə: $[0x47, 0x37, 0x94, 0xED]^T$.

11.3 Həll 3: Açar Genişləndirmə ($W[4]$)

$W[4]$ yeni raundun ilk sözüdür və $W[4] = W[0] \oplus g(W[3])$ düsturu ilə hesablanır. Verilənlər: $W[0] = [2B, 7E, 15, 16]$, $W[3] = [09, CF, 4F, 3C]$.

$g(W[3])$ funksiyasının tətbiqi:

1. **RotWord**: $[CF, 4F, 3C, 09]$
2. **SubWord**: $[S(CF), S(4F), S(3C), S(09)] = [8A, 84, EB, 01]$
3. **Rcon XOR**: $[8A \oplus 01, 84, EB, 01] = [8B, 84, EB, 01]$

Yekun XOR:

$$W[4] = [2B, 7E, 15, 16] \oplus [8B, 84, EB, 01] = [A0, FA, FE, 17]$$

11.4 Həll 6: Python Kodu İzahı

Təqdim olunan implementasiyada aşağıdakı məntiq izlənilir:

- **IV (Başlanğıc vektoru)**: Şifrələmənin hər dəfə unikal olması üçün `os.urandom(16)` istifadə edilir.
- **Padding**: AES 16-baytlıq bloklarla işləyir. Əgər mətn tam bölünmürsə, PKCS7 standartı ilə doldurulur.
- **CBC Rejimi**: Hər bir məlumat bloku şifrələnməzdən əvvəl özündən əvvəlki şifrəli blokla XOR edilir.

12 Nəticə

Bu mühazirə çərçivəsində müasir kriptografiyanın sütunlarından biri hesab olunan **AES (Advanced Encryption Standard)** alqoritmini həm nəzəri, həm də praktiki aspektdən təhlil etdik. Öyrəndiyimiz əsas məqamlar aşağıdakılardır:

- **AES-in Tarixi və Standartlaşma**: Rijndael alqoritminin DES-i əvəz edərək necə global standart halına gəlməsi (Wright, 2001).
- **Alqoritmin Arxitekturası**: Dəyişən raund saylı (10, 12, 14) Substitution-Permutation Network (SPN) strukturu və *State* matrisi (NIST, 2001).
- **SubBytes**: S-box vasitəsilə təmin edilən qeyri-xəttilik və diferensial hücumlara qarşı müqavimət (Daemen and Rijmen, 2002).

- **ShiftRows:** Sətirlərin dövrü sürüşdürülməsi ilə baytlararası qarışdırmanın (diffusion) təmini (Paar and Pelzl, 2010).
- **MixColumns:** $GF(2^8)$ meydanı üzərində matris vurulması ilə sütundaxili yüksək dərəcəli yayılma (Series, 2021).
- **AddRoundKey:** Gizli açarın raund açarları şəklində məlumatla birləşdirilməsi.
- **Key Expansion (Açarın Genişləndirilməsi):** Rcon sabitləri və qeyri-xətti funksiyalar vasitəsilə vahid açardan raund açarlarının törədilməsi (Stallings, 2017).
- **Təhlükəsizlik və Tətbiq:** Diferensial və xətti kriptanalizə dözümlülük, həmçinin TLS, Wi-Fi (WPA3) və disk şifrələmə sistemlərindəki kritik rolu.

Yekun olaraq qeyd edə bilərik ki, AES həm səmərəliliyi, həm də riyazi dözümlülüyü sayəsində yaxın gələcəkdə də (post-kvant dövrü daxil olmaqla) ən etibarlı simmetrik şifrələmə alqoritmi olaraq qalacaqdır.

Növbəti Mühazirə

Həftə 11: Blok şifrələrin iş rejimləri (Modes of Operation). ECB, CBC, CFB, OFB və CTR rejimlərinin təhlükəsizlik xüsusiyyətlərini və onların müxtəlif şəbəkə protokollarındakı tətbiqini araşdıracağıq.

References

- J. Daemen and V. Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Springer, Berlin, 2002. The authoritative guide to Rijndael by its designers [citation:4].
- M. Fowler. Aes-gcm-siv: Specification and analysis. *IACR Transactions on Symmetric Cryptology*, 2022(1):1–30, 2022.
- B. Gladman. Implementing aes. In *Cryptography and Coding*, pages 1–21. Springer, 2003.
- T. Good and M. Benaissa. Aes on fpga: Asip vs. picoblaze implementations. *Field-Programmable Logic and Applications*, pages 1–6, 2005.
- X. Inc. Aes encryption algorithms, 2019. Vitis Libraries Documentation [citation:1].
- J. Nechvatal, E. Barker, L. Bassham, W. Burr, M. Dworkin, J. Foti, and E. Roback. Report on the development of the advanced encryption standard (aes). *Journal of Research of the National Institute of Standards and Technology*, 106(3):511–577, 2001.
- NIST. Advanced encryption standard. Technical Report FIPS PUB 197, U.S. Department of Commerce, 2001.
- S. Oukili et al. High throughput fpga implementation of advanced encryption standard algorithm. *TELKOMNIKA*, 15(1):494–503, 2017.
- C. Paar and J. Pelzl. *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, Berlin, 2010. ISBN: 978-3642041006.
- C. S. Series. *Advanced Encryption Standard: Theory and Implementation*. Digital Education Press, 2021. Electronic Resource.
- W. Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 7 edition, 2017.
- M. Wright. The advanced encryption standard. *Network Security*, 2001(10):11–13, 2001. Announcement of Rijndael as AES winner [citation:2].